

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет
ИТМО»

Факультет программной инженерии и компьютерной техники



Операционные системы

Базовый трек

Лабораторная работа №2

Вариант: CLOCK

Выполнили:

Вильданов Ильнур Наилевич
Бондарев Алексей Михайлович
Группа №Р3312

Проверил:

Зубахин Дмитрий Сергеевич

Санкт-Петербург
2026 г.

Оглавление

Текст задания.....	2
Краткий обзор кода.....	4
Описание алгоритма вытеснения.....	4
Данные о работе программы-нагрузчика до и после внедрения своего page cache	5
Таблица 1. READ (случайное чтение).....	6
Таблица 2. WRITE (случайная запись)	6
Таблица 3. READ (последовательное чтение)	6
Таблица 4. WRITE (последовательная запись).....	7
Графики сравнения.....	7
Вывод.....	10

Текст задания

Для оптимизации работы с блочными устройствами в ОС существует кэш страниц с

данными, которыми мы производим операции чтения и записи на диск. Такой кэш позволяет избежать высоких задержек при повторном доступе к данным, так как операция будет выполнена с данными в RAM, а не на диске.

В данной лабораторной работе необходимо реализовать блочный кэш в пространстве пользователя в виде динамической библиотеки. Политику вытеснения страниц и другие элементы задания необходимо получить у преподавателя.

При выполнении работы необходимо реализовать простой API для работы с файлами, предоставляющий пользователю следующие возможности (по аналогии с системным API):

1. Открытие файла по заданному пути файла, доступного для чтения. Процедура возвращает некоторый хэндл на файл. Пример: ``int lab2_open(const char *path)``.
2. Закрытие файла по хэндлу. Пример: ``int lab2_close(int fd)``.
3. Чтение данных из файла.
Пример: ``ssize_t lab2_read(int fd, void buf[.count], size_t count)``.
4. Запись данных в файл.
Пример: ``ssize_t lab2_write(int fd, const void buf[.count], size_t count)``.
5. Перестановка позиции указателя на данные файла. Достаточно поддерживать только абсолютные координаты.
Пример: ``off_t lab2_lseek(int fd, off_t offset, int whence)``.
6. Синхронизация данных из кэша с диском. Пример: ``int lab2_fsync(int fd)``.

Операции с диском разработанного блочного кэша должны производиться в обход страничного кэша операционной системы.

В рамках проверки работоспособности разработанного блочного кэша необходимо адаптировать указанную преподавателем программу-загрузчик из ЛР 1, добавив использование кэша. Запустите программу и убедитесь, что она корректно работает. Сравните производительность до и после.

Вариант: Clock -- Часовой алгоритм.

Краткий обзор кода

vtpc.c – публичный API

vtpc_internal.h - внутренние структуры и константы

vtpc_cache_clock.c – кэш страниц и алгоритм CLOCK

vtpc_table.c - таблица соответствия vtpc_fd – state (внутреннее состояние: поиск/вставка/удаление)

vtpc_io.c - низкоуровневый I/O через системные вызовы

(open/pread/pwrite/fsync/close/lseek) и открытие с O_DIRECT когда требуется режим direct

vtpc_stats.c – сбор и печать метрик (hits/misses/evict/rd_pages/wb_pages/hit_rate)

io_load.c - нагрузчик: режимы cache=off и cache=on, замер времени, расчет IOPS/throughput

Описание алгоритма вытеснения

CLOCK – это эффективный алгоритм замещения страниц, приближение LRU.

Идея алгоритма:

- Есть фиксированное количество кадров (страниц кэша), страницы расположены по кругу
- Есть указатель clock hand (т.е. стрелка часов), которая показывает кандидата на вытеснение
- У каждой страницы есть ref бит обращения: если страницу недавно использовали, то ref=1; если давно не трогали, то ref=0

Как выставляется ref

- При любом обращении к странице (read/write hit, и после загрузки на miss) выставляется ref = 1
- Это означает, что страница нужна и не нужно ее выкидывать сразу

Когда кэш заполнен и пришел промах:

1. смотрим на страницу под clock_hand
2. если ref == 1:
 - даем второй шанс: ставим ref = 0
 - двигаем clock_hand дальше
3. если ref == 0:
 - это кандидат на вытеснение
 - если страница dirty, то делаем write-back (сбрасываем на диск)
 - помечаем страницу как невалидную (valid=0) и переиспользуем ячейку кэша под новую страницу
 - переиспользуем под новую страницу

И так по кругу, пока не найдем ref==0

Данные о работе программы-нагрузчика до и после внедрения своего page cache

Эксперименты выполнялись программой `io_load` с фиксированными параметрами нагрузки и изменяемым размером рабочего набора

Основные параметры:

- **rw=read/write** - тип операции: чтение/запись
- **block_size=4096** - размер одного запроса ввода-вывода (4 КБ, размер страницы)
- **block_count=80000** - количество операций (число выполненных блоковых чтений/записей)
- **type=random** - случайный доступ по диапазону range (имитация нерегулярных обращений к страницам)
- тестовый файл (использовался большой файл размером 1 Гб, чтобы размер файла не ограничивал выбор диапазона)

Параметры, задающие рабочий набор:

- **range=0-WS_BYTES** диапазон смещений, из которого выбираются обращения. Он ограничивает размер рабочего набора:
 - `ws_pages=32`; `WS_BYTES = 32 × 4096 = 131072`
 - `ws_pages=128`; `WS_BYTES = 128 × 4096 = 524288`
 - `ws_pages=256`; `WS_BYTES = 256 × 4096 = 1048576`

Параметры режима сравнения (до и после):

- **cache=off** – direct без vtpc
- **cache=on** – работа через vtpc (пользовательский page cache)

Отключение системного page cache ОС в direct:

- **direct=on** (для `cache=off`) - открытие файла с `O_DIRECT`, чтение/запись выполняются в обход системного page cache ОС.

Параметры VTPC:

- **VTPC_CACHE_PAGES=128** - емкость кэша VTPC (128 страниц = 512 КБ при 4 КБ/страница).
- **VTPC_LOG=1** - печать статистики кэша при закрытии файла (hits/misses/evict/hit_rate)

Повторяемость измерений:

- **repeat=5** - каждый эксперимент выполнялся 5 раз, в таблицы заносилось среднее значение

Таблица 1. READ (случайное чтение)

ws_pages	Direct IOPS (mean)	VTPC IOPS (mean)	Ускорение по IOPS	Direct throughput (MiB/s)	VTPC throughput (MiB/s)	Ускорение по thr	hit_rate	misses	evict
32	28950	3839581	132.63×	113.09	14998.36	132.63×	99.96%	32	0
128	29003	3031913	104.54×	113.29	11843.41	104.54×	99.84%	128	0
256	29039	56595	1.95×	113.43	221.07	1.95×	50.01%	39992	39864

При ws_pages=32 и ws_pages=128 рабочий набор влезает в кэш vtpc (128 страниц). Поэтому после короткого прогрева кэша число промахов равно размеру рабочего набора (misses=32 и misses=128), вытеснений нет (evict=0), а доля попаданий почти 100% (hit_rate≈99.8–99.96%). Соответственно это приводит к резкому росту производительности: ускорение примерно 105–133× (и по IOPS, и по throughput).

При ws_pages=256 рабочий набор в 2 раза больше кэша (256 > 128). В этом случае кэш начинает постоянно вытеснять страницы: вытеснений примерно 40 тыс., а hit_rate падает примерно до 50%, что совпадает с ожидаемой оценкой (capacity – емкость кэша) cap/ws = 128/256 = 0.5. Производительность vtpc в этом случае почти сравнивается с direct: ускорение падает до 1.95.

Таблица 2. WRITE (случайная запись)

ws_pages	Direct IOPS (mean)	VTPC IOPS (mean)	Ускорение по IOPS	Direct throughput	VTPC throughput	Ускорение по thr	hit_rate	misses	evict
32	26910	3299869	122.62×	105.12	12890.11	122.62×	99.96%	32	0
128	26948	2347157	87.10×	105.27	9168.58	87.10×	99.84%	128	0
256	26973	27665	1.03×	105.36	108.07	1.03×	49.91%	40073	39945

В целом, видны те же результаты, что и при случайном чтении: при ws ≤ 128 почти все операции попадают в кэш, следовательно получаем хорошее ускорение (87–123×), а при ws=256 вытеснения и существенное уменьшение ускорения

Таблица 3. READ (последовательное чтение)

ws_pages	Direct IOPS (mean)	VTPC IOPS (mean)	Ускорение по IOPS	Direct throughput (MiB/s)	VTPC throughput (MiB/s)	Ускорение по thr	hit_rate	misses	evict
32	28716	4482428	156.09×	112.17	17509.48	156.09×	99.96%	32	0
128	29096	3380774	116.19×	113.66	13206.15	116.19×	99.84%	128	0
256	28611	28126	0.98×	111.76	109.87	0.98×	0.00%	80000	79872

При ws_pages=256 рабочий набор превышает емкость кэша (256 > 128). В режиме последовательного доступа происходит циклический проход по диапазону, и повторное

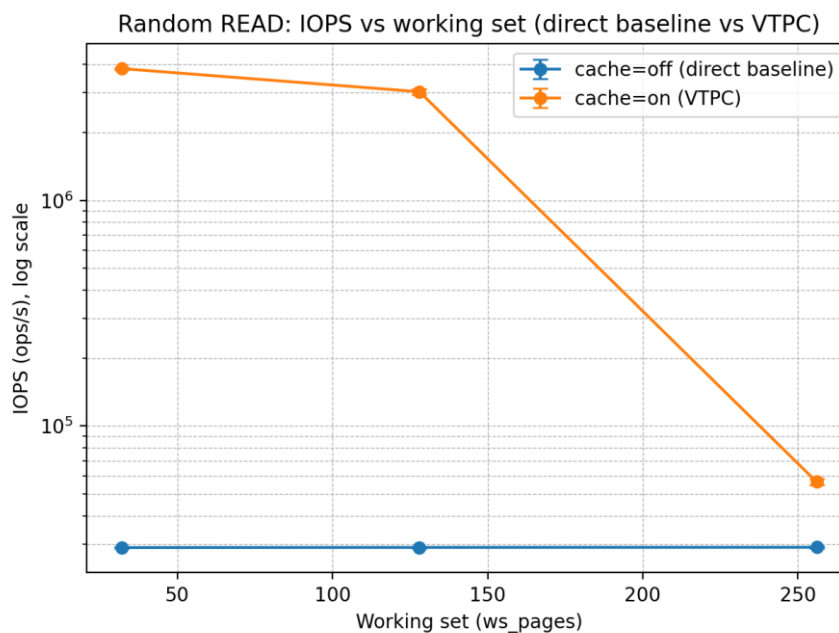
обращение к странице происходит только через 256 страниц, то есть позже, чем кэш способен удержать. Поэтому страницы вытесняются до повторного использования: $hit_rate \approx 0\%$, $misses \approx block_count$ (80000), $evict \approx 79872$. В результате VTPC не получает выигрыша от кэширования (ускорение $\approx 0.98\times$).

Таблица 4. WRITE (последовательная запись)

ws_pages	Direct IOPS (mean)	VTPC IOPS (mean)	Ускорение по IOPS	Direct throughput (MiB/s)	VTPC throughput (MiB/s)	Ускорение по thr	hit_rate	misses	evict
32	27530	4086766	148.45×	107.54	15963.93	148.45×	99.96%	32	0
128	27263	2871586	105.33×	106.50	11217.13	105.33×	99.84%	128	0
256	27156	13932	0.51×	106.08	54.42	0.51×	0.00%	80000	79872

При $ws_pages=256$ рабочий набор превышает емкость кэша ($256 > 128$) и в режиме последовательного доступа повторного использования страниц фактически нет ($hit_rate \approx 0\%$, $misses \approx 80000$, $evict \approx 79872$). При этом vtpc несет накладные расходы на ведение кэша и вытеснения; для записи дополнительно возникает запись на диск при вытеснении грязных страниц. Поэтому производительность становится ниже direct (ускорение $\approx 0.51\times$).

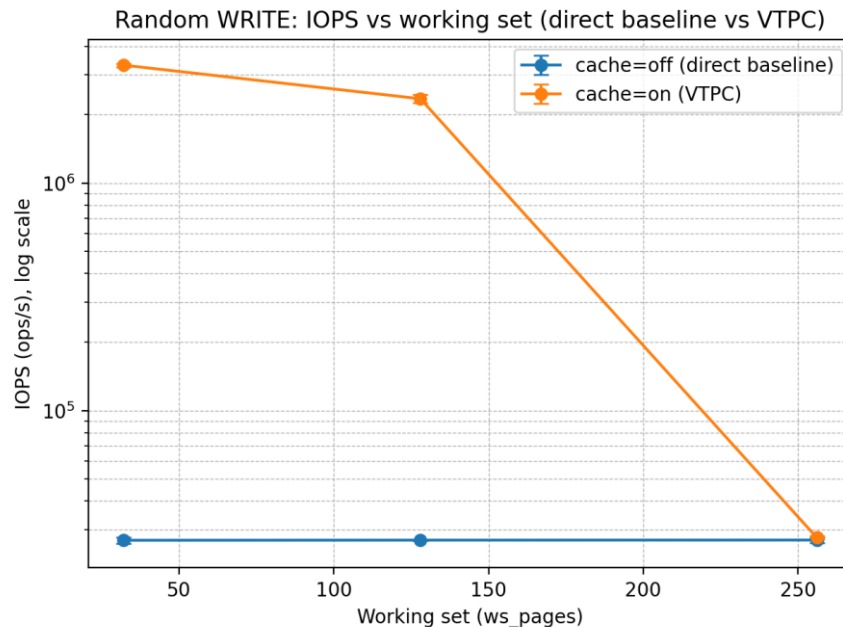
Графики сравнения



При $ws_pages=32$ и $ws_pages=128$ производительность vtpc на порядки выше direct. При увеличении ws от 32 к 128 ускорение сохраняется большим, но становится меньше.

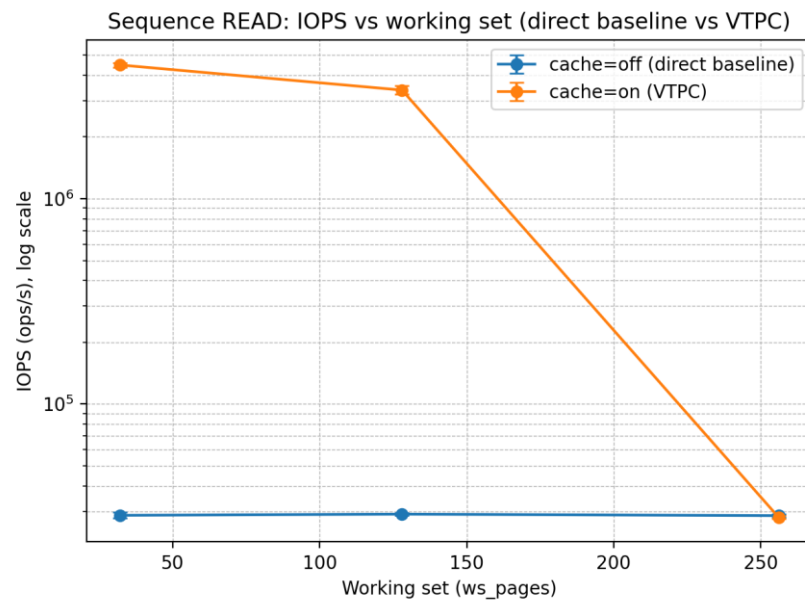
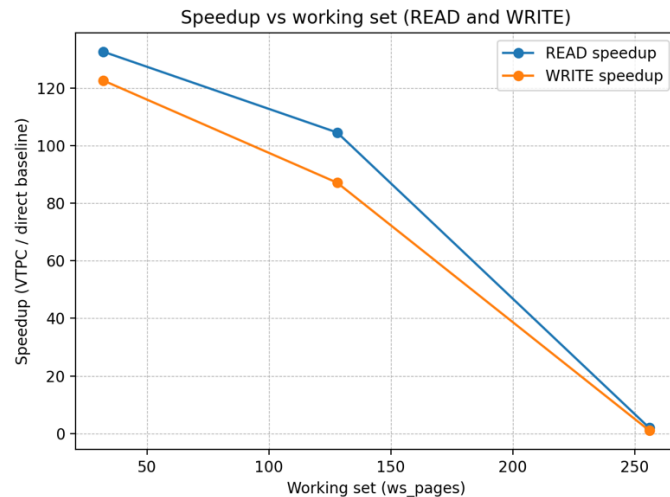
- direct (cache=off, direct=on) это диск: каждое обращение выполняется через системные вызовы чтения/записи с `O_DIRECT`, то есть в обход системного page cache ОС. Поэтому производительность ограничена IOPS подсистемы хранения
- VTPC (cache=on) - это пользовательский page cache. Когда рабочий набор полностью помещается в кэш ($ws_pages \leq cap=128$):
 - промахи происходят на холодной фазе прогрева (примерно $misses \approx ws_pages$)

- вытеснений практически нет ($\text{evict}=0$)
- далее подавляющее большинство операций - это попадания ($\text{hit_rate} \approx 99.8\text{--}99.96\%$)
- значит дорогостоящий ввод-вывод с диска происходит только на первых промахх, а потом чтение/запись обслуживается из памяти процесса
- Поэтому VTPC показывает ускорение на два порядка (в десятки–сотни раз), что и видно по IOPS/throughput



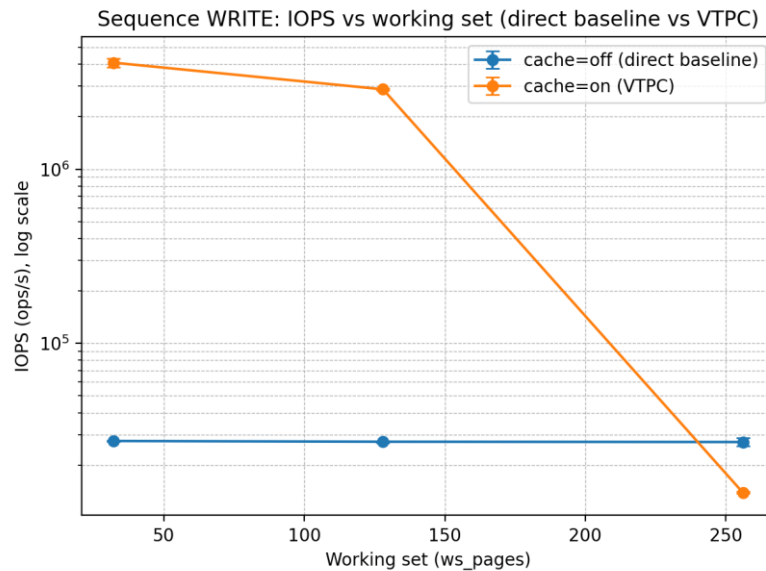
При $\text{ws_pages}=256$ разрыв между vtpc и direct резко сокращается и сравнивается в одной точке: ускорение становится умеренным (порядка единиц), а иногда почти пропадает.

- При $\text{ws_pages}=256$ рабочий набор в 2 раза больше емкости кэша ($256 > 128$). В этом случае кэш не может удержать весь набор страниц одновременно
- Начинается интенсивное вытеснение и повторные загрузки страниц: страницы успевают вылететь до того, как их снова попросят
- подтверждается статистикой VTPC:
 - evict становится большим (десятки тысяч)
 - hit_rate падает примерно к теоретическому $\text{cap/ws} = 128/256 \approx 50\%$
- В результате VTPC вынужден регулярно ходить на диск, и по сути частично возвращается к дисковому режиму, поэтому ускорение падает

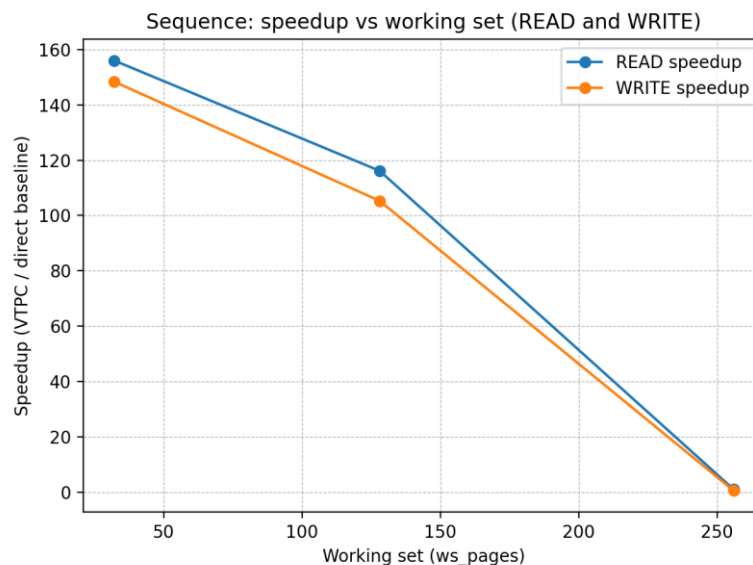


При `ws_pages=32` и `ws_pages=128` линия `vtpc` значительно выше `direct`, при `ws_pages=256` разрыв резко сокращается и становится даже чуть ниже.

- `direct (cache=off, direct=on)`: каждое обращение идет через `direct I/O` (мимо `page cache` ОС), поэтому скорость ограничена `IOPS` хранения.
- `vtpc (cache=on)` при `ws_pages ≤ cap=128`: почти все обращения превращаются в попадания, а промахи в основном на прогреве (`misses ≈ ws_pages`), затем почти одни попадания (`hit_rate ≈ 100%`), вытеснений нет (`evict=0`), `IOPS` на порядки выше.
- При `ws_pages=256` в режиме `sequence` повторного использования страниц почти нет; кэш не удерживает страницу до следующего обращения (`hit_rate ≈ 0%`, `misses ≈ block_count`, `evict` большой), `vtpc` часто ходит на диск и деградирует к `direct`.



При последовательной записи по рабочему набору, превышающему емкость кэша ($ws_pages=256 > cap=128$), повторного использования страниц практически нет ($hit_rate \approx 0\%$), поэтому кэш не дает выигрыша и превращается в накладной расход. Из-за затрат на управление кэшем и частые вытеснения производительность vtpc становится ниже direct ($speedup < 1$).



Вывод

В ходе лабораторной работы была реализована библиотека пользовательского кэширования страниц для файлового ввода-вывода. Реализация обеспечивает корректную работу основных операций чтения, записи, перемещения позиции в файле и синхронизации данных. Для управления заполнением кэша используется алгоритм замещения CLOCK. Работоспособность решения подтверждена тестами и запуском программы-нагрузчика, а собранные метрики показали, что кэширование эффективно ускоряет доступ к данным в случаях повторного обращения к одним и тем же областям файла, при этом поведение при недостатке памяти кэша соответствует ожидаемому: возрастает число промахов и вытеснений, что снижает эффективность.